# Data Flow Diagram

## CrediShield AI — Credit Card Approval Prediction System

---

### Level 0 — Context Diagram

The context diagram shows the system as a single process interacting with external entities.

```
                                   Application Data (JSON)
┌───────────────────┐         ──────────────────────────────────▶    ┌──────────────────────────┐
│                   │                                                │                          │
│   User /          │                                                │   CrediShield AI         │
│   Credit Analyst  │                                                │   Prediction System      │
│                   │                                                │                          │
│                   │         ◀──────────────────────────────────    │                          │
└───────────────────┘         Prediction Result (JSON) + PDF         └──────────────────────────┘
                                                                                  │
                                                                                  │ Reads
                                                                                  ▼
                                                                     ┌──────────────────────────┐
                                                                     │   Training Datasets      │
                                                                     │   (application_record +  │
                                                                     │    credit_record CSVs)   │
                                                                     └──────────────────────────┘
```

**External Entities:**

- **User / Credit Analyst**: Submits applicant data and receives predictions
- **Training Datasets**: Kaggle CSV files used during model training phase

---

### Level 1 — System Decomposition

The system is decomposed into four major processes.

```
┌──────────────────────────────────────────────────────────────────────────┐
│                          CrediShield AI System                           │
│                                                                          │
│   ┌─────────────────┐     ┌─────────────────┐     ┌────────────────┐     │
│   │  P1: Web UI     │────▶│  P2: Flask API  │────▶│ P3: ML Engine  │     │
│   │  (Presentation) │     │  (Controller)   │     │  (Prediction)  │     │
│   │                 │◀────│                 │◀────│                │     │
│   │  - index.html   │     │  - app.py       │     │ - model.joblib │     │
│   │  - style.css    │     │  - /predict     │     │ - scaler       │     │
│   │  - script.js    │     │  - /metrics     │     │ - encoders     │     │
│   │  - Chart.js     │     │  - /            │     │ - features     │     │
│   └─────────────────┘     └─────────────────┘     └────────────────┘     │
│                                    │                                     │
│                           ┌────────▼────────┐                            │
│                           │ P4: Data Store  │                            │
│                           │ (Metrics/Stats) │                            │
│                           │                 │                            │
│                           │ - metrics.json  │                            │
│                           │ - stats.json    │                            │
│                           └─────────────────┘                            │
│                                                                          │
└──────────────────────────────────────────────────────────────────────────┘
```

**Process Descriptions:**

| Process | Name | Input | Output | Technology |
|---|---|---|---|---|
| P1 | Web UI (Presentation Layer) | User interactions (clicks, form entries) | HTTP requests to Flask API | HTML5, CSS3, JavaScript, Chart.js |
| P2 | Flask API (Controller Layer) | HTTP requests (JSON payloads) | Processed predictions, metrics data | Flask 3.0.3, Python 3.12 |
| P3 | ML Engine (Prediction Layer) | 18-element feature vector | Prediction (0/1) + probability | Scikit-learn, Gradient Boosting |
| P4 | Data Store (Persistence Layer) | Training outputs | Metrics and statistics for charts | JSON files |

---

### Level 2 — Detailed Prediction Data Flow

This diagram shows the step-by-step data transformations inside the `/predict` endpoint.

```
User Input (14 fields via JSON POST)
    │
    │  {gender, age, income, education, family_status, children,
    │   housing_type, occupation, years_employed, family_size,
    │   own_car, own_realty, income_type, unemployed}
    │
    ▼
┌──────────────────────────────────────┐
│  STEP 1: Input Validation            │
│  - Check all 14 required fields      │
│  - Return 400 error if any missing   │
└──────────────────────────────────────┘
               │
```

```
                 | ☑ All fields present
                 ▼
┌───────────────────────────────────────┐
│ STEP 2: Parse Numerical Values        │
│ - age → int                           │
│ - income → float                      │
│ - children → int                      │
│ - family_size → int                   │
│ - years_employed → float              │
│ - unemployed → int (0 or 1)           │
└───────────────────────────────────────┘
                 │
                 ▼
┌───────────────────────────────────────┐
│ STEP 3: Feature Engineering           │
│ - income_per_member = income /        │
│   family_size                         │
│ - income_cat: Low/Medium/High         │
│   (thresholds: $100K, $200K)          │
│ - age_cat: Young/Middle-Aged/Senior   │
│   (thresholds: 30, 50)                │
│ - employed_cat: Unemployed/Junior/    │
│   Mid-level/Senior                    │
│   (thresholds: 0, 5, 15 years)        │
└───────────────────────────────────────┘
                 │
                 ▼
┌───────────────────────────────────────┐
│ STEP 4: Categorical Encoding          │
│ LabelEncoder.transform() for:         │
│ - CODE_GENDER (M/F → 0/1)            │
│ - FLAG_OWN_CAR (N/Y → 0/1)           │
│ - FLAG_OWN_REALTY (N/Y → 0/1)         │
│ - NAME_INCOME_TYPE (5 categories)     │
│ - NAME_EDUCATION_TYPE (5 cats)        │
│ - NAME_FAMILY_STATUS (5 cats)         │
│ - NAME_HOUSING_TYPE (6 cats)          │
│ - OCCUPATION_TYPE (19 cats)           │
│ - INCOME_CAT (3 categories)           │
│ - AGE_CAT (3 categories)              │
│ - EMPLOYED_CAT (4 categories)         │
│ Total: 11 encoded features            │
└───────────────────────────────────────┘
                 │
                 ▼
┌───────────────────────────────────────┐
│ STEP 5: Numerical Scaling             │
│ StandardScaler.transform() for:       │
│ - CNT_CHILDREN                        │
│ - AMT_INCOME_TOTAL                    │
│ - CNT_FAM_MEMBERS                     │
│ - AGE                                 │
│ - YEARS_EMPLOYED                      │
│ - INCOME_PER_MEMBER                   │
│ Total: 6 scaled features              │
└───────────────────────────────────────┘
                 │
                 ▼
┌───────────────────────────────────────┐
│ STEP 6: Feature Vector Assembly       │
│ - Read feature_names.joblib           │
│ - Arrange all 18 features in the      │
│   exact order used during training    │
│ - Create numpy array: shape (1, 18)   │
└───────────────────────────────────────┘
                 │
                 ▼
┌───────────────────────────────────────┐
│ STEP 7: Model Inference               │
│ - model.predict(features)             │
│   → 0 (Rejected) or 1 (Approved)      │
│ - model.predict_proba(features)       │
│   → [prob_rejected, prob_approved]     │
└───────────────────────────────────────┘
                 │
                 ▼
┌───────────────────────────────────────┐
│ STEP 8: Risk Classification           │
│ If Approved:                          │
│   confidence = prob_approved × 100    │
│   ≥ 85% → Low Risk                    │
│   ≥ 65% → Medium Risk                 │
│   < 65% → High Risk                   │
│ If Rejected:                          │
│   confidence = prob_rejected × 100    │
│   risk_level = "High"                 │
```

```
└──────────────────────────────────────┘
                   │
                   ▼
┌──────────────────────────────────────┐
│  STEP 9: JSON Response                │
│  {                                    │
│    "approved": true/false,            │
│    "confidence": 98.76,               │
│    "risk_level": "Low"                │
│  }                                    │
└──────────────────────────────────────┘
                   │
                   ▼
    Web UI renders:
    - APPROVED/REJECTED badge
    - Animated SVG confidence gauge
    - Risk level indicator
    - PDF download button
    - Saves to localStorage history
```

---

## Data Store Descriptions

| Data Store | File | Contents | Created By | Used By |
|---|---|---|---|---|
| DS1 | `card_model.joblib` | Trained GradientBoostingClassifier (136 KB) | `train.py` | `app.py` (prediction) |
| DS2 | `scaler.joblib` | Fitted StandardScaler object (1 KB) | `train.py` | `app.py` (numerical scaling) |
| DS3 | `label_encoders.joblib` | Dictionary of 11 LabelEncoder objects (3.5 KB) | `train.py` | `app.py` (categorical encoding) |
| DS4 | `feature_names.joblib` | Ordered list of 18 feature column names (0.3 KB) | `train.py` | `app.py` (vector assembly) |
| DS5 | `metrics.json` | Model comparison scores for all 6 models (3.8 KB) | `train.py` | `app.py` → `/metrics` endpoint |
| DS6 | `stats.json` | Training dataset statistics (1.1 KB) | `train.py` | `app.py` → `/metrics` endpoint |
| DS7 | `localStorage` | Prediction history log (browser-side) | `script.js` | `script.js` (history tab) |

---

**Project Name:** CrediShield AI
**Author:** Dinesh (dineshTech-creator)
**Date:** July 2026